

RESEARCH ARTICLE

Cloudhive: A Cloud-Based Framework for Smart Grid Co-Simulation, Data, and Communication

Kenneth B. Kent¹ | Mengbing Zhou^{2,3} | Gabriel Adeyemo¹ | Yang Wang^{2,3}

¹Faculty of Computer Science, University of New Brunswick, Fredericton, Canada | ²Shenzhen Institutes of Advanced Technology, Chinese Academy of Sciences, Shenzhen, China | ³University of Chinese Academy of Sciences, Beijing, China

Correspondence: Yang Wang (yang.wang1@siat.ac.cn)

Received: 15 January 2025 | **Revised:** 22 July 2025 | **Accepted:** 23 July 2025

Funding: This work was supported by The Atlantic Innovation Fund (AIF) program and The Chinese Academy of Sciences President's International Fellowship Initiative (Grant No. 2023VTA0001).

Keywords: big data | cloud computing | co-simulation | message-oriented middleware | smart grid

ABSTRACT

The integration of renewable energy has driven the need for smart grid frameworks that enable efficient co-simulation, data management, and secure communication. This paper introduces CloudHive, a cloud-native framework designed to address these challenges by unifying large-scale power-network co-simulation, real-time data communication, and big data analytics in a single modular architecture. Unlike existing co-simulation tools or data platforms that operate in isolation, CloudHive uniquely enables bidirectional interaction between simulation environments (e.g., OpenDSS for power systems, OMNeT++ for communication networks) and real-world smart grids, supported by message-oriented middleware (RabbitMQ, Apache Kafka) for low-latency data exchange and Kubernetes for dynamic scalability. We evaluate CloudHive's accuracy, scalability, and usability through three representative case studies. The results show that CloudHive achieves high accuracy, performs well in real-world scenarios, and scales efficiently with growing workloads in cloud environments.

1 | Introduction

Electric power generation, distribution, and demand management are rapidly evolving due to advances in renewable energy sources and related technologies. In traditional grids, information flow is unidirectional, control is centralized, and most connected devices and agents—particularly demand-side elements—are passive. In contrast, a smart grid enables bidirectional information and energy flow, decentralizes energy sources, activates demand, and distributes control. These advancements necessitate substantial modifications to existing power networks, including upgrades to the underlying information and communication technology (ICT) infrastructure, data communication networks, and the integration of smart

meters, sensors on low- and medium-voltage lines, devices, and substations [1].

A key feature of the smart grid is the integration of ICT with power grids. This integration underscores two key points. First, smart grids are complex systems comprising various interdependent domains. The overall performance of the grid relies on the collective performance of these domains and their interactions. As a result, smart grid designs must undergo joint verification and validation through simulations before implementation [2]. While testing these systems in real-world environments is impractical, available simulators may not support the simulation of all relevant domains. For instance, power system simulators often fail to model communication infrastructures, and vice versa.

Abbreviations: MOM, message-oriented middleware; PDC, phasor data concentrator; PMU, phasor measurement units.

Developing a new simulator to model both power system dynamics and communication networks is costly and time-consuming [1]. Co-simulation offers a more cost-effective solution by integrating multiple simulation environments that exchange data to create a collaborative system [3, 4]. A co-simulation framework allows for the simultaneous simulation of power system dynamics and communication networks, reducing both time and cost. However, the primary challenges of co-simulation include ensuring time synchronization across simulators, enabling efficient data exchange, and managing the time ratio—the simulation time relative to wall clock time.

Second, the introduction of bidirectional information flow and an increasing number of connected active devices results in a massive volume of data that must be managed across the smart grid [5, 6]. This data, characterized as big data, can provide valuable insights for both consumers and utility companies. However, managing these data sources—along with their integration, aggregation, analytics, and storage—presents significant challenges. The data is voluminous, constantly changing, and typically in diverse formats, requiring real-time processing for reactive control and monitoring. A system capable of handling this data must be secure, scalable, highly available, and equipped with mechanisms for fast processing, storage, and retrieval.

Therefore, this paper explores CloudHive, a framework designed to support three core functions: (1) Efficiently handling data communication in smart grid co-simulation, (2) enabling communication between simulation environments and real systems, and (3) securely managing the aggregation, analytics, and storage of big data in smart grid systems.

Unlike traditional co-simulation tools or isolated data platforms, CloudHive uniquely integrates power-network and communication co-simulation with real-world grid systems through hybrid middleware (RabbitMQ, Apache Kafka) and cloud-native orchestration (Docker, Kubernetes), enabling bidirectional data flow and dynamic scalability. It addresses the gap in holistic data management by unifying real-time streaming (Apache Kafka), analytics (Apache Spark), and secure storage (MongoDB) with end-to-end RSA encryption, while its modular design supports empirical validation across scenarios like demand-response applications. This makes CloudHive a scalable, secure, and practically validated framework for bridging smart grid simulation, communication, and big data challenges, with adaptability for future advancements.

The remainder of this paper is structured as follows: Section 2 provides background information essential to understanding the design of CloudHive, while Section 3 reviews the communication technologies and techniques used in the CloudHive framework. Section 4 describes its implementation, and Section 5 presents test case implementations and evaluation results. Finally, we conclude with a discussion of our findings and identify opportunities for future research.

2 | Background

In this section, we provide an overview of power system simulation, communication system simulation, and their co-simulation.

This background knowledge is essential for understanding the design of our simulation system.

2.1 | Power System Simulation

Power system dynamic simulation is typically modeled as a continuous-time system simulation, meaning it is time-driven. In a continuous-time system, state variables evolve continuously with respect to time. The system dynamics are typically described by a set of differential equations, which define the transitions between state variables [7]. These equations can be solved analytically in simple cases or using numerical methods for more complex systems [1]. In numerical simulations, the system's behavior is captured through an iterative process. A typical iteration loop involves calculating network boundary variables, followed by the calculation of secondary variables. This process continues until a preset stop time is reached. When visualized along a time axis, the simulation consists of discrete iterations at small time intervals.

Open Distribution System Simulator (OpenDSS) is an open-source tool for simulating power distribution systems [1]. It serves as a differential equation solver for power generation and load dynamics, handling both harmonics and time-domain analysis. This well-documented simulator supports a variety of load types, photovoltaic systems, on-load tap changers, energy storage systems, and other distribution network components [8, 9]. OpenDSS allows users to define circuits, execute control commands on circuit elements, configure simulation parameters, and visualize or export results. It supports time-stepped power flow simulation and integrates with other programming languages like C++, Python, C#, and MATLAB via a COM interface.

2.2 | Communication System Simulation

Data communication, an essential part of the smart grid, is often modeled and tested using discrete-event simulation techniques. Discrete-event-driven simulation is ideal for systems where state changes occur only due to discrete events, such as the transmission of messages or packets across the network. Events are managed in an event queue, with each event triggering the execution of corresponding actions, which may, in turn, generate additional events. The event queue and events themselves are managed by an event scheduler, which maintains the event timeline and determines the sequence in which events are processed.

OMNeT++ is an open-source, modular C++ framework primarily used for modeling communication networks, multiprocessors, and distributed systems [10, 11]. It supports various communication protocols, including TCP, IP, Ethernet, Wi-Fi, and several networking applications, making it a versatile tool for simulating communication systems [3].

2.3 | Co-Simulation

Most commercially available power system simulators do not support modeling communication infrastructures, and vice versa. As a result, co-simulation—combining the strengths of individual simulators—is a practical solution. Li et al. [1] present

three possible approaches to co-simulation. The first approach involves building new software capable of modeling both domains. However, this option is costly and time-consuming. The second approach extends the functionality of individual simulators to support the other domain. While this is less expensive, it is typically limited to simpler scenarios and becomes inefficient when power system dynamics or communication networks are too complex [12]. The third and most viable option involves integrating existing simulators to form a co-simulation framework. Since power system simulators are time-driven and communication network simulators are event-driven, the primary challenge in co-simulation is efficient synchronization of data, time, and interactions between the simulators. A successful co-simulation framework must address issues such as the common data model, which integrates components from all domains, and the choice of a suitable time step for synchronization [13]. Li et al. [1] and Mirz et al. [13] highlight three basic synchronization techniques:

Time-stepped synchronization, where each domain-specific simulator runs independently, and the simulators are paused and synchronized at specified time intervals. During simulation, events requiring interaction—such as data requests or control command execution—are stored and resolved during synchronization. This approach is fast but susceptible to time accumulation errors based on the chosen simulation time step.

Primary-Secondary synchronization, where one simulator (either the power system or communication system) acts as the Primary, coordinating co-simulation steps. This method is simple to implement, but challenges arise in adapting the time resolution of both simulators to ensure accurate message exchanges [12].

Global Event-Driven synchronization, where a global event list is maintained, containing events from both the power system and communication system. These events are executed according to their timestamps. Power system simulation, while time-driven, is discrete and treated as special events in the event queue [7]. The downside of this approach is slower simulation speed, as each simulator must pause to resolve every event [1].

2.4 | Data Transmission

Data transmission involves the mechanisms and services responsible for moving data between different points across the grid. Many devices have incompatible interfaces, so network protocols are used to enable communication between them. These protocols define a set of rules that allow devices with different interfaces to communicate over a network. In this study, we consider two lightweight protocols: The Advanced Message Queuing Protocol (AMQP) and the Message Queuing Telemetry Transport (MQTT). Both protocols are well-suited for smart grid environments and constrained devices. They enable clients to communicate with middleware brokers using a publish/subscribe model. Clients can publish messages to specific topics and subscribe to messages from one or more topics. The broker ensures that messages are routed to the correct destinations [14]. These protocols were selected for their performance, reliability, support for publish/subscribe, and compatibility with other framework tools, over alternatives such as the Constrained Application Protocol (CoAP) [15], Extensible Messaging and

Presence Protocol (XMPP) [16], and Data Distribution Service (DDS) [17].

3 | Framework Design

The design of the information and communication structure in a smart grid system relies on its architectural design and functional requirements. To address the diverse aspects of this structure, a wide range of tools and technologies are available. In developing an efficient and comprehensive framework (CloudHive) to support co-simulation, communication, and big data management, we evaluate various technologies across each layer of the grid's information and communication structure. The evaluation focuses on criteria such as flexibility, scalability for large workloads, high performance, availability, fault tolerance, security, and support for real-time simulation and streaming data analytics. Figure 1 presents the complete design of this framework.

3.1 | Time Synchronization Techniques

To achieve seamless co-simulation with minimal temporal inconsistencies, CloudHive adopts a hybrid time synchronization scheme that integrates the primary-secondary and time-stepped synchronization techniques. In this setup, the communication simulator OMNeT++ serves as the primary controller, while the power simulator OpenDSS operates as the secondary. The synchronization mechanism proceeds as follows:

- *Time Step and Synchronization Interval*: The simulation advances in discrete time steps, which define the temporal granularity and determine the simulation speed in the absence of interrupting events. A configurable synchronization interval is used to periodically align both simulators when no critical event is detected, ensuring loosely coupled progress with bounded drift.
- *Primary-Secondary Interaction*: OMNeT++ (primary) coordinates the simulation by issuing requests to OpenDSS (secondary) for circuit solving, data retrieval, control action

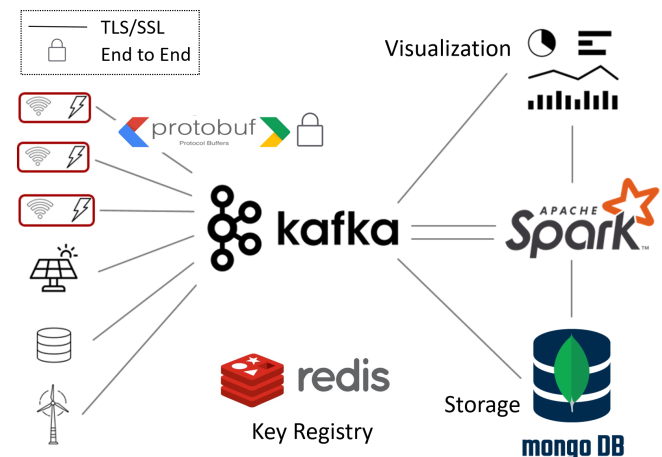


FIGURE 1 | Co-simulation Framework. The red-bordered rectangle represents a co-simulation instance with power and communication simulators.

execution, or synchronization. The two simulators operate under a unified time scale to maintain consistency.

- *Event Classification and Handling:* Events in OMNeT++ are dynamically classified as either critical or non-critical. Critical events, such as control commands, faults, or errors, are time-sensitive and trigger immediate synchronization to ensure timely and accurate system responses. In contrast, non-critical events like status queries or information updates are less urgent and can be processed asynchronously or deferred until the next scheduled synchronization point.
- *Synchronization Triggers:* A synchronization point is initiated either periodically at the end of each synchronization interval (when no critical events occur) or immediately upon detection of a critical event. At each synchronization point, OMNeT++ sends a message to OpenDSS, instructing it to solve the power flow for a specified time window, thereby aligning the simulation states of both systems.

This hybrid mechanism effectively avoids error accumulation common in pure time-stepped approaches by responding to critical events in real time, while also mitigating time resolution mismatches typical of traditional primary-secondary synchronization. Additionally, to enable integration with real-time systems, CloudHive replaces the sequential scheduler with a real-time scheduler that aligns the simulation clock with the physical clock.

3.2 | Data Communication Middleware

In co-simulation environments, selecting an optimal data exchange method is crucial for reducing communication overhead. While the impact of suboptimal communication is minimal in small-scale scenarios, it becomes significant as the model scales, affecting simulation time and accuracy.

Middleware facilitates communication within the co-simulation environment and with external systems, including real-world smart grid systems. It ensures efficient data and control signal exchange between devices, applications, and grid sections.

For smart grids, Message-Oriented Middleware (MOM) is ideal due to its ability to handle event-based data exchange and support both asynchronous and synchronous communication. A subtype of MOM, Publish-Subscribe middleware, decouples publishers and subscribers, allowing them to function asynchronously and independently.

In the *CloudHive* framework, we utilize RabbitMQ, an open-source message broker supporting multiple messaging protocols. RabbitMQ ensures high availability and efficient message exchange through a publish-subscribe mechanism, making it a suitable solution for distributed communication in the smart grid environment.

3.3 | Data and Message Format

The middleware abstracts communication but does not define the message structure. To ensure efficient data exchange, a common

data format must be agreed upon, and each party must be able to serialize and deserialize data in that format. The chosen format must be flexible, lightweight, cross-platform, and efficient to meet the smart grid's dynamic and high-speed requirements.

For this, we use Protocol Buffers (protobuf) [18] and JSON. Protocol Buffers are a binary-based format that is efficient, compact, and fast, suitable for high-performance scenarios. JSON is lightweight and human-readable, ideal for simpler data exchanges. The choice of format impacts message size, performance, and serialization efficiency.

To handle evolving data, we incorporate schema versioning, tagging messages with schema versions to ensure compatibility between old and new data formats. A schema registry tracks and manages these versions.

In this design, each message consists of a header and a body. The header includes metadata (such as schema version and encryption details), while the body contains the serialized data. This separation allows flexibility in serialization formats and facilitates schema evolution. For example, the header can be serialized in JSON for simplicity, while the body uses Protocol Buffers for efficiency.

3.4 | Data Aggregation and Analysis

To leverage the large streams of data in the smart grid, it must be efficiently collected, analyzed, and visualized. The services must be fast, efficient, secure, and scalable to meet the needs of the smart grid. Stream processing is employed to apply logic to each new data instance as it arrives, ensuring timely and accurate results compared to batch processing [19]. This approach aligns well with the dynamic needs of the smart grid environment.

For data aggregation, Apache Kafka is used to gather data from various nodes across the smart grid, including simulation instances. Kafka is a distributed event streaming platform that supports high-performance data pipelines, streaming analytics, and data ingestion. It handles high availability, fault tolerance, and scalability, making it well-suited for the smart grid's data needs. Kafka integrates well with various data sources and sinks, including IoT devices, databases, and analytics tools like Apache Spark.

For data analytics, we use Apache Spark, which integrates seamlessly with Kafka for processing streaming data. Spark is a distributed computing engine that enables parallel data processing across clusters. It supports a variety of storage systems and provides a flexible framework for handling large-scale data processing tasks. This setup allows efficient data analysis and real-time insights, which are crucial for the smart grid framework.

3.5 | Data Storage

The storage of data for future analysis, auditing, error tracing, and historical reference is crucial in many smart grid systems. Data is written and read at high speeds, and thus, fast, efficient, and scalable data storage systems are required to meet these demands.

NoSQL databases are commonly employed for data storage in the smart grid due to their scalability and superior performance in handling big data [20–22]. A NoSQL database stores data with a flexible schema, in contrast to the fixed columns and rows of relational databases. An experiment by E. Yilmaz et al. [23] comparing different storage options for the smart grid concluded that Apache Cassandra and MongoDB outperform HBase and SQL options like PostgreSQL. For our framework, we chose MongoDB as the NoSQL database solution.

3.6 | Data Security

In the smart grid environment, data can include not only electrical parameters but also sensitive user consumption data and proprietary company information. To ensure data integrity, privacy, and confidentiality, we secure the smart grid data in two primary ways: During transmission (in transit) and while stored (at rest).

For securing data in transit, we employ Transport Layer Security (TLS) and point-to-point encryption (P2PE) protocols. These technologies provide secure communication between directly connected systems, ensuring that data is protected from interception and tampering during transmission. In our framework, TLS/SSL is used to secure data exchanges between services. The management of TLS certificates is handled via Kubernetes Secrets to store and manage sensitive information.

For data at rest, we use end-to-end encryption to ensure that data is only accessible by the sender and the receiver, preventing unauthorized access by intermediaries or attackers. We use the RSA asymmetric encryption algorithm [24] to encrypt data at rest, with the public keys of services managed centrally through a Redis key-value store and the private keys handled by Kubernetes Secrets. This approach ensures secure management of encryption keys across services in the framework.

3.7 | Cloud-Based Environment

To accommodate the dynamic needs of the smart grid, the framework leverages cloud computing to provide scalable, on-demand

resources. Each service within the framework is containerized using Docker, allowing them to run in isolated environments while sharing underlying resources. These containers are managed and orchestrated by Kubernetes, an open-source platform that automates deployment, scaling, and management of containerized applications [25]. Kubernetes organizes containers into logical units, simplifies management, and ensures efficient resource allocation.

Kubernetes also handles service scaling, security management, and configuration. It stores sensitive data in Kubernetes Secrets and manages configuration variables through ConfigMap, allowing for dynamic service configurations. All services in the framework are containerized and deployed in a cloud environment, enabling seamless scaling and resource management.

4 | Framework Implementation

CloudHive is designed to support both real-world smart grid and simulation scenarios, offering flexibility in configuration and tool substitution based on specific use cases. For instance, Apache Kafka can replace RabbitMQ as a message broker in co-simulations where higher throughput is needed and complex routing is not a priority. This paper presents a microservice-based implementation of CloudHive, as shown in Figures 2 and 3. The key details of the implementation are as follows:

- *KafkaClient and KafkaAdminClient*: The *KafkaClient* class provides an abstraction for interacting with Kafka, offering methods to send and receive serialized messages between services and Kafka. It manages message serialization and deserialization to ensure seamless data transfer across services. The *KafkaAdminClient* is responsible for managing Kafka resources, such as creating and configuring topics, managing partitions, and performing administrative tasks. Both classes are based on the open-source Python Confluent Kafka library (v1.7.0), and they can be easily configured through environment variables to adapt to different deployment settings and use cases.

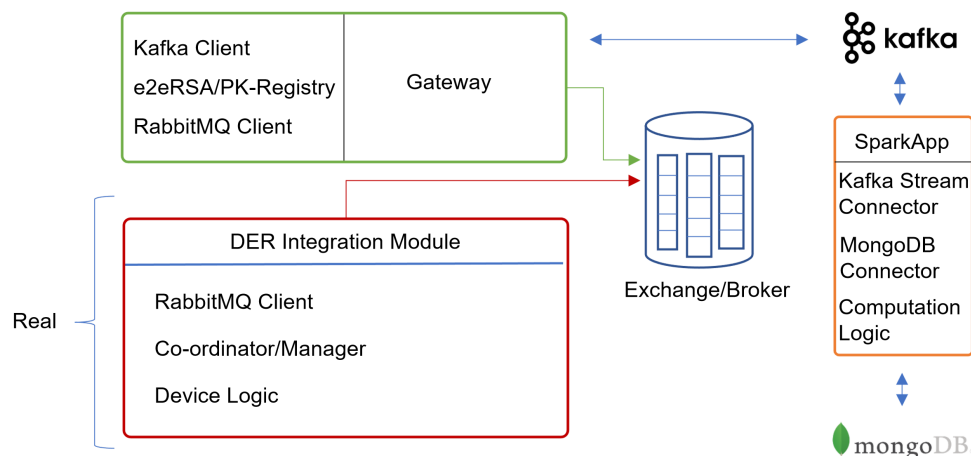


FIGURE 2 | Microservice implementation of a cloud-based framework with real devices.

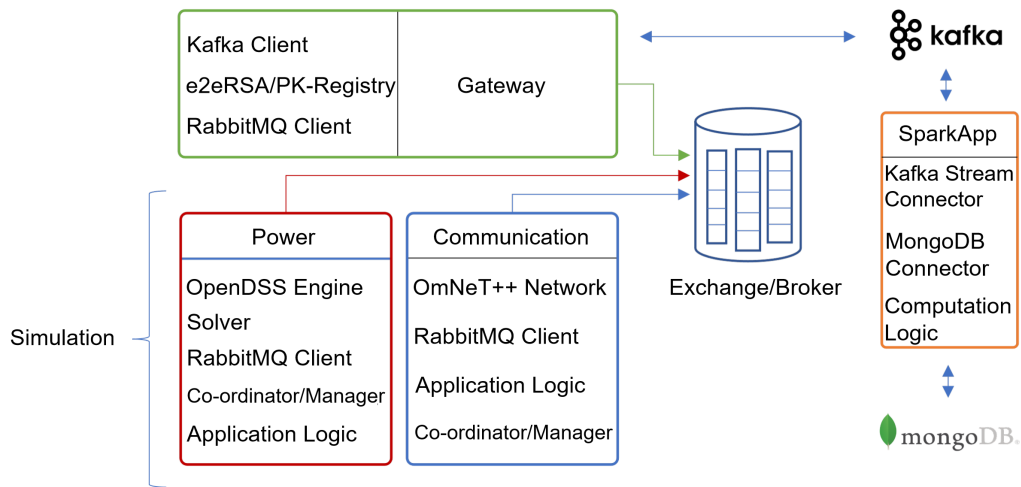


FIGURE 3 | Microservice implementation of a cloud-based framework for simulation.

- *RabbitMQClient*: This class abstracts communication with RabbitMQ, providing methods for routing messages to queues and receiving messages from them. It also includes methods for creating, configuring, and managing exchanges and queues. The C++ version of this client, used in OmNeT++, is built on the Copernica Marketing Software AMQP-CPP open-source library [26]. The Python version of the class utilizes the Pika library (v1.2.0) [27]. CloudHive leverages RabbitMQ's Quorum queues [28], which operate on the Raft consensus algorithm, ensuring high availability and reliability for message delivery and handling even in the event of node failures.
- *E2eRSAEncryption & PublicKeyRegistry*: The *E2eRSAEncryption* class provides end-to-end asymmetric encryption for messages exchanged between services, ensuring secure communication within the system. This class offers methods for generating private and public key pairs, or for retrieving PEM-formatted keys stored in the environment. It works in conjunction with the *PublicKeyRegistry* class, which manages the distribution and storage of public keys for different services, enabling secure key exchange. The implementation is based on the Pyca Cryptography library (v3.4.8) [29], a well-known open-source Python library for cryptographic operations.
- *Gateway Service*: The gateway service acts as an interface between the simulation or integrated devices and the aggregation layer, facilitating communication between these components without requiring changes to the services interacting with the aggregation layer. This design allows easy substitution of simulation or real devices. The gateway also ensures a bi-directional flow of data and commands, enabling seamless control across the system. Importantly, the gateway ensures data localization, meaning that only data that has passed through the gateway is made available outside the simulation or integration module, as shown in Figures 2 and 3. While this design supports both real and simulation scenarios, the current implementation focuses on the architecture depicted in Figure 2, with plans to explore integration with Distributed Energy Resources (DERs) in future work.
- *OpenDSS Solver*: The OpenDSS Solver, integrated into the system, models and solves power circuits within the co-simulation environment. It interacts with the OpenDSS COM interface through the *dss-python* (v0.10.7.post1) library, enabling the execution of power flow simulations and solving electrical circuits. This functionality is essential for smart grid simulations, where real-time power flow analysis and system modeling are required.
- *Co-ordinator/Manager*: This module is responsible for interpreting incoming messages, events, and commands, and for passing control to the appropriate application logic. The logic of the application depends on the specific co-simulation use case or the system's goals, which are defined by the developer. This module serves as the control center for managing the flow of events and ensuring the correct execution of the system's processes.
- *Host Startup Service*: The host startup service is responsible for the initial setup of the system. It initializes the environment, sets necessary environment variables, creates Kafka topics and RabbitMQ queues, generates encryption keys, and ensures that all resources required for the system to run are correctly initialized and bootstrapped. This containerized service ensures that all dependencies and configurations are handled automatically during system startup.
- *Spark Application*: The Spark application is a central component for performing complex parallel computations on large datasets. It is designed to handle computationally intensive tasks based on the specific use case defined by the developer. The Spark application includes connectors that retrieve data from external data sources, such as databases and Kafka streams, apply computational logic, and write the processed results back to the storage systems. The connectors utilized in this implementation include *kafka-sql-streaming-2.1.1:3.1* for streaming data and *mongo-spark-connector-2.12:3.0.1* for

MongoDB integration, enabling seamless data processing across various platforms.

5 | Experiments

CloudHive is deployed on a three-node Kubernetes cluster running on multi-core servers. Each node is connected to the Center for Advanced Studies-Atlantic laboratory network and assigned a static IP address. The cluster is initialized using Kubeadm (v1.18.4) [30], ensuring consistent and scalable service deployment. Each node operates on Ubuntu 16.04.06 LTS, equipped with six Intel i7-8700 cores, 32 GiB of memory, and 500 GiB of disk storage, providing a robust environment for running CloudHive at scale.

We evaluate the performance and feasibility of CloudHive through a series of experiments based on the IEEE 13-bus Feeder Circuit [31]. The experiments include co-simulation of the power and communication functions of the circuit. Key variables include message payload size, the number of instances, message formats, and message generation rates. Each experiment is executed at least 10 times, excluding warm-up and cold-start runs.

The evaluation metrics include accuracy, latency, messages processed per second, message size, scalability, and time ratio. The time ratio refers to the ratio of wall-clock time to simulation time. For example, with a Real-Time Scheduler scaled 400 times, without application logic and requests, the time ratio is 0.0025. This means it takes 9 s to simulate one hour (3,600 s). The time ratio directly influences latency and throughput: A smaller time ratio indicates a faster simulation, enabling more messages to be processed and sent per second.

5.1 | Accuracy

In this test case, we compare the accuracy of a co-simulation with and without CloudHive to ensure that the simulation results remain consistent. Three monitors, M1, M2, and M3, are placed at corresponding nodes in the 13-bus circuit, with each monitor represented in the OmNeT++ network. The monitors are synchronized and sampled every 15 simulation minutes. The simulation is conducted with the communication simulator as the primary and the power simulator as the secondary. Voltage at each node is measured over 24 simulation hours. We then compare the results of the simulation with CloudHive against those without it.

Initially, we sample the voltage at the monitors without co-simulation to establish a baseline for comparison. Next, we simulate the communication network using a sequential event scheduler, allowing the primary simulator to run continuously without pausing for synchronization. This setup incorporates the latency effects introduced by co-simulation. As shown in Figure 4, the voltage readings in both scenarios exhibit identical trends, confirming that the simulation's accuracy is preserved. However, latency is observed between the synchronization request and the receipt of the corresponding voltage reading at the primary simulator. The recorded time t_x at the primary simulator is expressed as:

$$t_x = t_{\text{time of request}} + \text{latency}$$

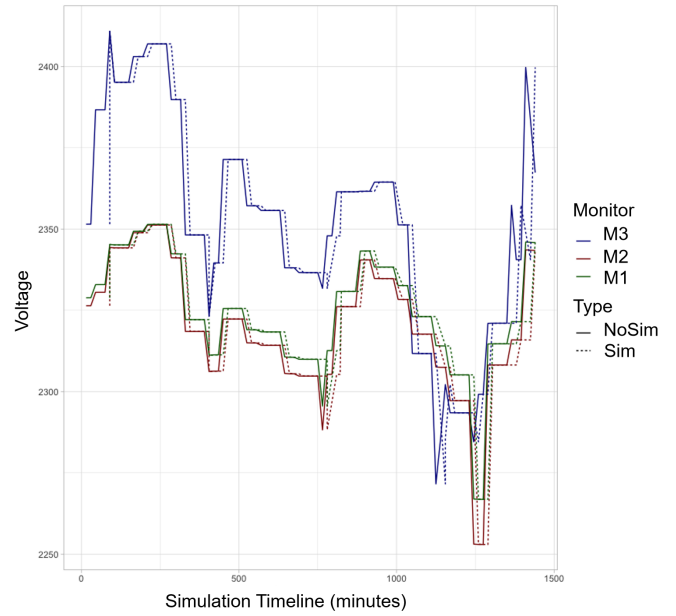


FIGURE 4 | Without vs. With Co-simulation (Sequential Scheduler): Voltage reading at 15-minute intervals for 24 hours.

where latency is the time delay between the request and the response. This emphasizes the importance of minimizing latency in co-simulation.

The following conclusions can be drawn regarding latency in co-simulation:

- If the focus is on accuracy, and the simulation is not time-sensitive (e.g., non-real-time or without hardware in the loop), latency becomes less significant. In this case, the simulation may proceed without stringent real-time requirements, and the primary simulator can tolerate slight delays in synchronization.
- For time-sensitive or real-time simulations, especially those involving hardware in the loop (e.g., real DERs), the Real-Time Scheduler, which can scale and synchronize with wall-clock time, should be used. This ensures that the simulation is in sync with real-world timing.
- With the Sequential Scheduler, the primary simulator may be paused at each synchronization point until the request receives a response from the secondary simulator. This method, illustrated in Figure 5, ensures that the voltage readings (represented by triangle points) in the co-simulation align exactly with the simulation without CloudHive. However, this method can introduce delays and increase the overall simulation time.

In all cases, the power readings and voltage waveforms in both simulations are consistent, confirming the accuracy of CloudHive in maintaining simulation integrity.

5.2 | Scalability: Wide Area Network Monitoring

B. Dhananjay et al. designed a co-simulator for a wide-area monitoring network of Phasor Measurement Units (PMUs) [3].

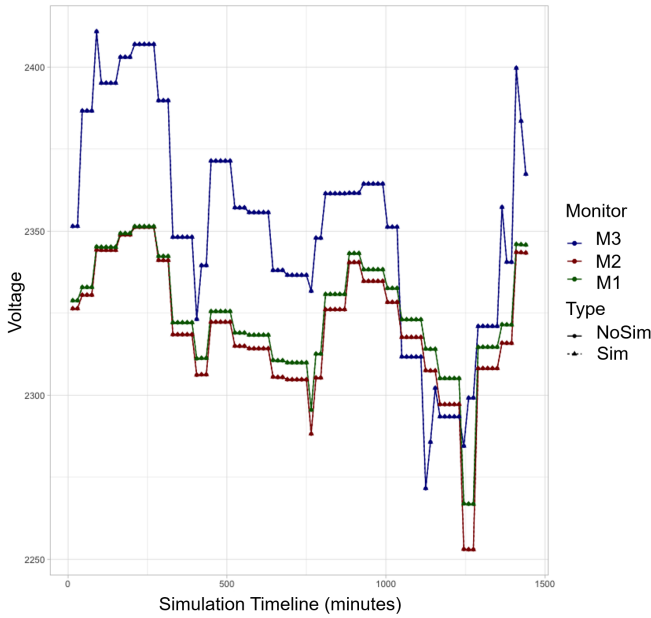


FIGURE 5 | Co-simulation with pausing and Sequential Scheduler: Voltage reading at 15-minute intervals for 24 hours.

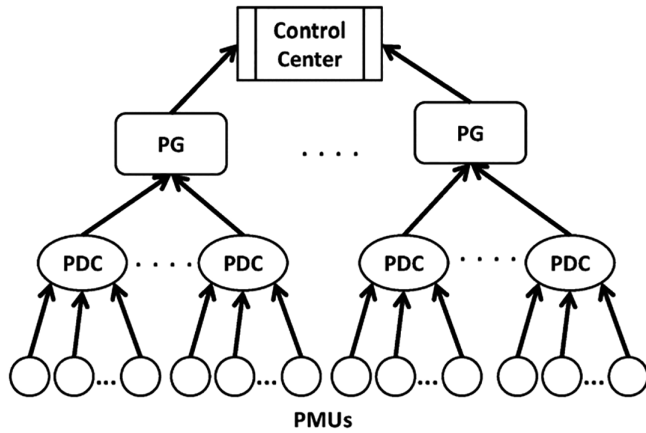


FIGURE 6 | Wide Area Monitoring Network [3].

Figure 6 shows the architecture of the network. In their co-simulator, the PMU data is stored in a database by the power circuit and queried by timestamp. Each PMU fetches the data at specified intervals, generating the corresponding PMU packet to be sent to the Phasor Data Concentrator (PDC). Each PDC processes the data and forwards it to the Phasor Gateway. Dhananjay et al. [3] compared the running time of the wide-area monitoring system with varying PMU generation rates over a one-minute simulation. The network consists of 10 PMUs, 4 PDCs, and 2 Phasor Gateways. The authors set the synchronization interval between the two simulators to 10 s. One significant bottleneck in their implementation is the large latency associated with fetching PMU readings at fixed intervals. For example, if data is fetched at five-second intervals, a value produced just after the previous fetch cycle is not delivered until the next cycle, resulting in a latency of at least four seconds.

In CloudHive, we implement the wide-area monitoring system, replacing the database polling from the benchmark

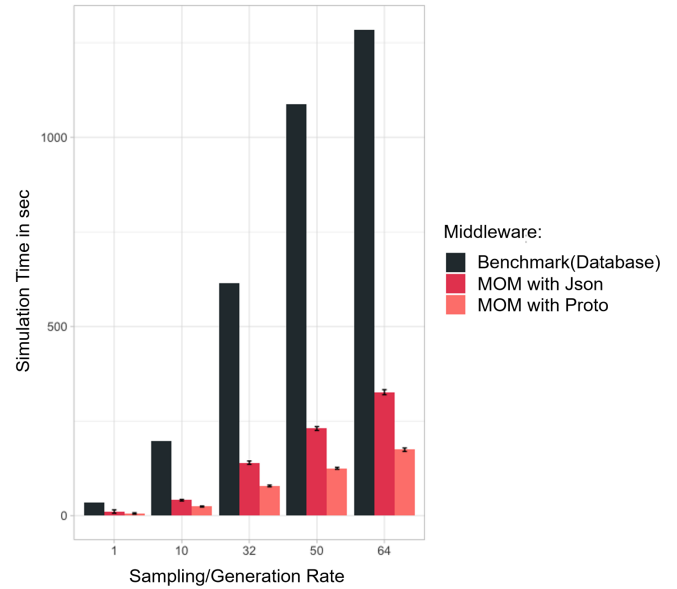


FIGURE 7 | Time required to simulate one minute of simulation time with Message-oriented and Database Middleware.

implementation with message-oriented middleware. We increase the number of PMUs to 12, with three PMUs attached to each PDC. Although the exact message payload size in the benchmark system is unknown, we choose to send approximately 100 voltage values per message, resulting in a message size between 1061 and 2400 bytes when serialized. According to a study by C. Rodriguez et al. [32], the median size of JSON payloads in REST-API systems is about 1545 bytes. We assume that the message size would be smaller for IoT devices. We measure the average real-time required to simulate one minute at each generation rate across multiple runs, comparing our results with the benchmark implementation [3]. As shown in Figure 7, both versions of our co-simulation framework, based on message-oriented middleware, outperform the system with database polling. The performance difference is significant, with the queue-based middleware exhibiting greater efficiency. The difference in performance is also influenced by the aggregation nature at each level of the wide-area monitoring network. Additionally, we observe that serialization using a binary-based Protocol Buffer format outperforms JSON serialization at this payload size.

Next, using the same implementation, we investigate the effect of higher workload and throughput on the average latency of messages to evaluate the scalability of CloudHive. Latency is defined as the response time from the data request to the response delivery, which includes both the round-trip message time and the processing time at the destination. We vary the PMU data generation rate from one message per PMU per second to 64 messages per PMU per second. For example, at a sampling rate of 50, the total messages per second reach 600. The results, shown in Figures 8 and 9, reveal the following:

- The results confirm that, across the varied sampling rates, the message-oriented middleware with Protocol Buffers performs better than JSON, achieving lower latency.

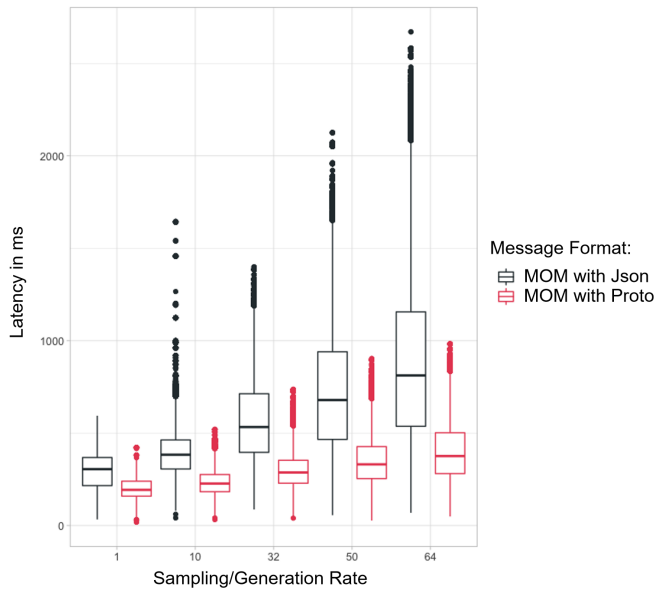


FIGURE 8 | Message latency with varied messages per second.

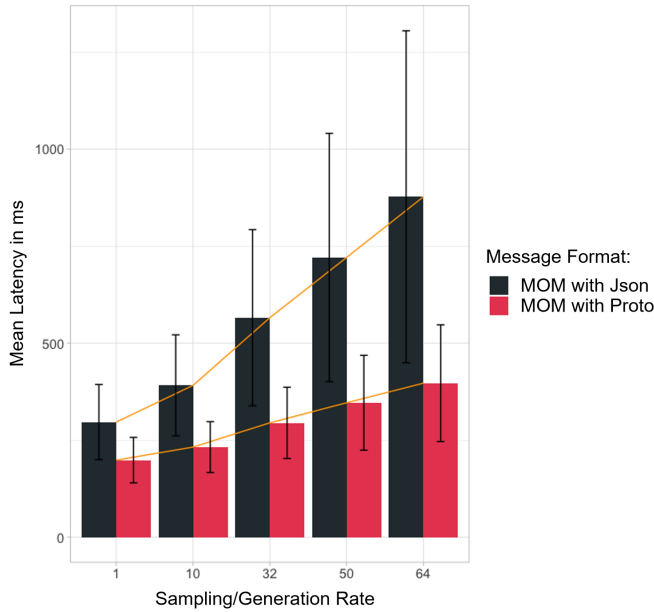


FIGURE 9 | Average latency of Message-oriented Middleware (MOM) with JSON and Protocol Buffer across varied messages per second.

- b. The relationship between message latency and throughput (messages per second) is linear. As the workload increases, the average latency increases accordingly.
- c. Although the latency-throughput relationship is linear (Figure 9), the increase in latency with respect to throughput is more pronounced in the JSON-based system.
- d. The standard deviation of the latency, represented by error bars, is large. The deviation ranges from ± 50 to 150 ms, likely caused by factors such as processing time at the data request target, network fluctuations, garbage collection in the middleware, message flushing to disk, or queue data

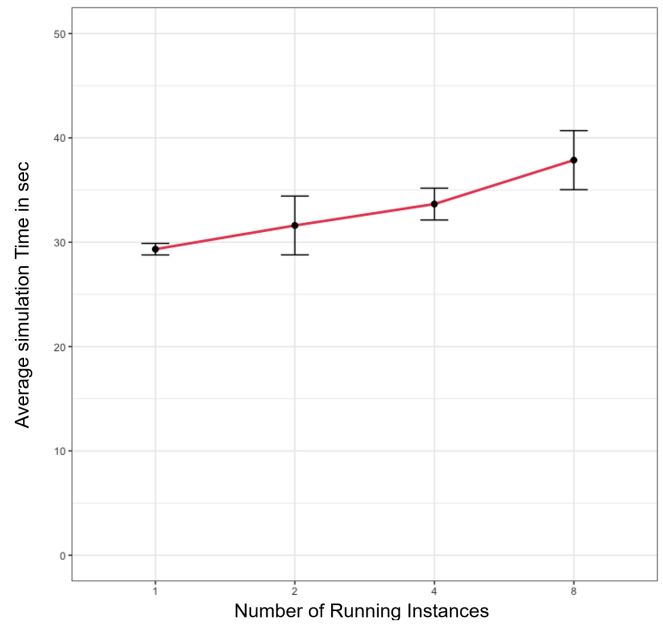


FIGURE 10 | Average real time to simulate one minute with varying numbers of running instances at 10 samples per second.

replication. The Protocol Buffer solution exhibits more stability, with fewer outliers, as shown in Figure 8. We explore this further in Section 5.2.2.

5.2.1 | Scaling

To evaluate the scalability of CloudHive, we run multiple instances of the wide-area monitoring co-simulation simultaneously and observe their impact on performance. Each instance simulates one minute of simulation time with a PMU sampling rate of 10. We scale the system by running two to eight instances simultaneously. As the number of instances increases, the number of messages per second rises. For instance, with two simultaneous instances at a sampling rate of 10, the system processes 240 messages per second. The results, shown in Figure 10, reveal that the average simulation time increases as more instances are run concurrently. The standard deviation of the simulation time also increases with multiple instances but remains within ± 5 s. The message-oriented middleware efficiently handles the increased message load as the number of messages per second grows. In contrast, in a database polling system, the size of the database table increases with more instances, which slows down query lookups as the system filters a larger dataset. In a queue-based message-oriented system, only unconsumed messages in the queue are processed, avoiding empty round trips to the data source when no new data is available.

5.2.2 | Message Size

Numerous studies have reported varying message sizes for smart meter data packets: T. Shiobara et al. report 150 bytes [33], B. Karimi et al. report sizes between 20 to 500 bytes [34], D. Hartmann et al. report sizes from 20 to 2133 bytes [35], and M. Jaber et al. reports a maximum of about 145,000 bytes [36]. To

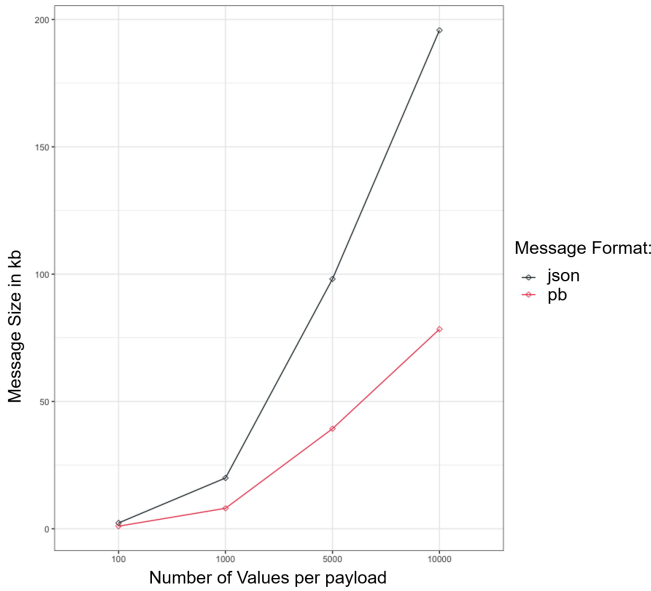


FIGURE 11 | Effect of number of values in a message payload on Message size.

accommodate different smart meter or grid device scenarios, we consider varying message sizes in our experiments. As demonstrated in previous sections, the Protocol Buffer-based solution performs better than JSON in message-oriented middleware systems. Therefore, we investigate the effect of message size on scalability by varying the number of voltage values in the payload and observing the impact on simulation time. In Figure 11, we show how the message size changes with 100 to 10,000 voltage values per payload. Overall, we find that JSON messages tend to have larger sizes than Protocol Buffer messages. Furthermore, the increase in message size is less pronounced with Protocol Buffers, suggesting that they may be more efficient in terms of storage for large-scale systems. However, in the results shown in Figure 12, we observe the following:

- At a sampling rate of 10 (120 messages per second), simulation time is longer with JSON-based messages when there are 100 values per message, but it improves as the number of values per message increases.
- For larger numbers of messages per second, Protocol Buffers outperform JSON in terms of simulation time, especially from 100 to 10,000 messages per second. A steep increase in simulation time is observed with JSON-based messages when the message rate reaches 1000 messages per second. This is likely due to the larger average change in message size with increasing sample numbers. Additionally, based on the design of RabbitMQ used for message-oriented middleware, the processing time in each handler is influenced by the size of the payload (a larger payload takes longer to process).

In CloudHive, we conclude that while JSON has advantages in specific scenarios, Protocol Buffers is the preferred choice due to its superior performance under higher message rates and reduced storage requirements. Furthermore, the results suggest that at high throughputs, sending multiple smaller response messages is more efficient than transmitting a single large message. For

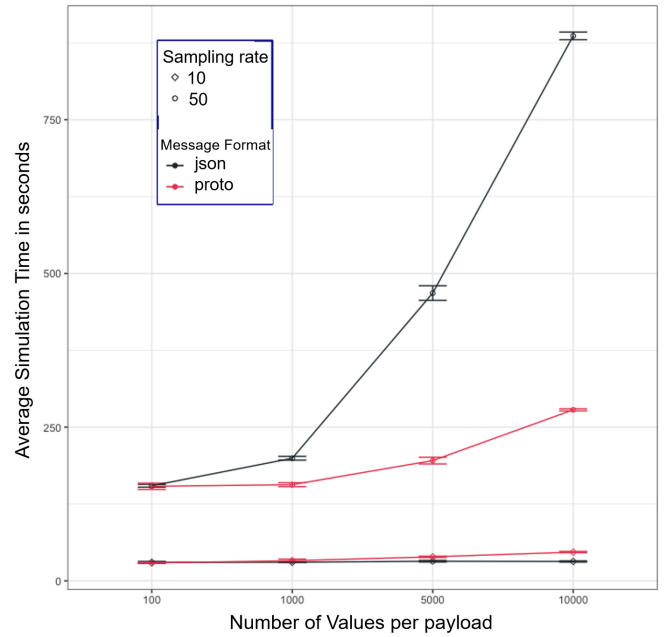


FIGURE 12 | Effect of message size on Simulation Time.

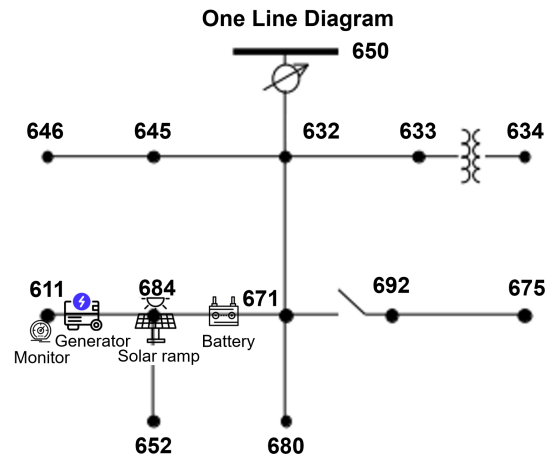


FIGURE 13 | A modified IEEE 13-bus feeder with generator and a monitor.

instance, splitting a dataset into two messages of 5000 values each outperforms a single message containing 10,000 values.

5.3 | Scalability: Cloud-Based Demand Response

This experiment demonstrates CloudHive's ability to support cloud-based analytics and bidirectional control in a co-simulation environment. We modify the benchmark circuit by adding a generator and a voltage monitor at Load 611 (Figure 13). The voltage at this bus is monitored and synchronized through the communication network. These data are then streamed via the gateway and aggregator to a Spark Analytics application in the cloud.

The Spark application applies a control model to the real-time voltage data to determine whether the generator should be

TABLE 1 | A sample dataset for a minimum and maximum threshold logistic regression model.

Voltage	Label	Command
2400	0	Nothing
2349	1	Generator ON
2451	2	Generator OFF

toggled ON or OFF. The control model is either trained from historical data or loaded from MongoDB. Although the control logic could be implemented using simple threshold-based rules, we adopt a logistic regression model as a proof-of-concept to demonstrate CloudHive's ability to incorporate flexible, data-driven analytics and support real-time closed-loop control in smart grid scenarios.

5.3.1 | Modeling the Logistic Regression Model

The control strategy used here is intentionally kept simple and is not intended to represent an optimal control method. Instead, it showcases how CloudHive supports cloud-based training and deployment of machine learning models for real-time decision-making.

Specifically, the logistic regression model predicts control actions based on voltage deviations from a base voltage of 2450 kV, with thresholds of ± 50 . We generate a training dataset containing voltage readings and corresponding control actions: 0 for Do Nothing, 1 for ON, and 2 for OFF (Table 1). This data is stored in MongoDB and used by the Spark application to train the model through multiple iterations to ensure basic classification accuracy.

5.3.2 | Results

The experiment demonstrates that the framework supports bi-directional data and command flows in a cloud environment, a crucial feature for modern smart grid simulations and real-world systems. Additionally, we showcase a simple application in demand-response systems. The results, shown in Figure 14, first display the voltage or load shape at Load 611 without the co-simulation or cloud-based demand-response. A voltage drop below the threshold occurs between 100 s and 160 s. With the demand-response application in co-simulation, the logistic algorithm controller processes the real-time voltage readings and detects the low voltage, triggering the command to activate the generator. Once the generator is started, the voltage is restored and maintained within the threshold. The bi-directional flow of data and commands in the presented framework is not limited to simple scenarios, such as toggling a generator to balance voltage in a power flow analysis. It also supports more complex functions, including fault resolution in the power circuit model during fault studies or adjusting power simulation parameters based on inputs through the communication simulator.

6 | Future Work

Real-world smart grid applications extend far beyond demand response systems to include power flow analysis, customer

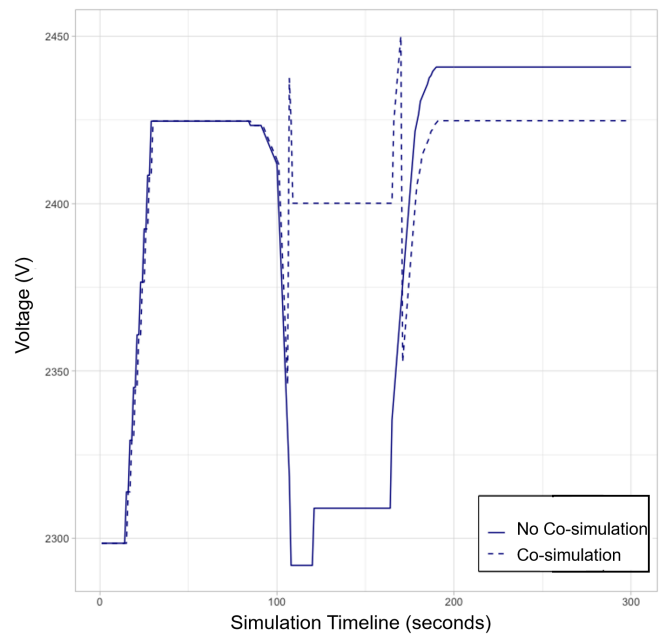


FIGURE 14 | Voltage at monitor with cloud-based demand-response using the analytics component.

behavior modeling, fault studies, and advanced grid management. While our analytics evaluation utilized a simple predictive control model, future work could explore complex machine learning techniques such as Support Vector Machines, K-Means clustering, and Gradient Boosting to enhance decision-making capabilities. As computational complexity increases, the processing time may affect latency and response time. To address this, future work could incorporate windowed queries, which perform computations over time windows, helping to maintain performance in more complex scenarios.

The middleware and analytics tools in our framework are highly configurable, providing opportunities for performance optimization based on specific use cases. For future evaluations, it would be beneficial to use real-world datasets instead of synthetic data, as this would better reflect the challenges and requirements of practical smart grid systems.

The framework's design supports the integration of real-life systems, facilitating communication between simulations and physical devices. However, future efforts should focus on exploring Distributed Energy Resource (DER) integration, particularly addressing time resolution discrepancies across hardware and simulators. While our use of OmNeT++'s Real-Time Scheduler offers a foundation for synchronization, its limited multi-threading capabilities may pose bottlenecks for larger-scale simulations. This could be improved in future versions of the framework.

For data security, our current framework employs RSA encryption for end-to-end security, but its computational overhead and ciphertext expansion pose challenges for high-throughput scenarios. A hybrid approach combining AES for data encryption and RSA for key exchange could mitigate these issues. AES offers superior performance for bulk data encryption, while

RSA ensures secure key distribution. However, this introduces complexity in key management and device compatibility, which we plan to address through lightweight key storage mechanisms (e.g., Redis for public keys and Kubernetes Secrets for private keys) in future iterations.

Finally, while our framework was evaluated on an on-premise cluster, future work should explore cloud deployments on platforms such as AWS, Google Cloud, and IBM Cloud. The framework's design ensures reproducibility with minor configuration changes, making it adaptable to various cloud environments.

7 | Conclusion

In this paper, we highlighted the distinctions between conventional grids and advanced smart grids, emphasizing the importance of information and communication technologies in the latter. Two major challenges were identified: (1) The need for efficient co-simulation frameworks that facilitate communication between simulations and real systems, and (2) the need for effective management of large volumes of data in smart grid environments.

To address these challenges, we proposed a cloud-centric, scalable framework—CloudHive—that supports:

1. Efficient data communication in smart grid co-simulation.
2. Bi-directional flow of data and commands between simulation and real systems.
3. Secure aggregation, analytics, and storage of smart grid data.

We designed, implemented, and evaluated the framework by comparing serialization formats (JSON and Protocol Buffers) and demonstrating the scalability, accuracy, and functionality of the system through three co-simulation use cases based on a benchmark IEEE circuit. The evaluation validated the framework's accuracy and scalability in cloud environments, as well as its ability to handle a wide range of workloads.

Our results showed that while JSON outperformed Protocol Buffers in some scenarios, Protocol Buffers proved to be the superior choice for higher message rates and smaller message sizes. Additionally, we demonstrated the flexibility of the analytics component by implementing a demand-response application, where a logistic regression model maintained voltage stability at a power node. This illustrated the framework's ability to support cloud-based bi-directional data and command flow.

In summary, CloudHive fulfills its design objectives and provides a robust foundation for advancing research and development in smart grid co-simulation and real-time applications. While the current framework establishes a comprehensive architectural base, future work will aim to quantify the performance trade-offs of its modular components—e.g., assessing the impact of different security protocols in high-throughput, low-latency scenarios. This will help delineate the practical boundaries of the framework and guide further optimization and adoption in real-world deployments.

Acknowledgments

This research was conducted within the Center for Advanced Studies—Atlantic, Faculty of Computer Science, University of New Brunswick. The authors are grateful to the colleagues and facilities of CAS—Atlantic for supporting our research. The authors would like to acknowledge the funding support provided by the Atlantic Canada Opportunities Agency (ACOA) through the Atlantic Innovation Fund (AIF) program and the Chinese Academy of Sciences President's International Fellowship Initiative (Grant No. 2023VTA0001).

Data Availability Statement

The authors have nothing to report.

References

1. W. Li, M. Ferdowsi, M. Stevic, A. Monti, and F. Ponci, "Cosimulation for Smart Grid Communications," *IEEE Transactions on Industrial Informatics* 10, no. 4 (2014): 2374–2384.
2. D. Oudart, J. Cantenot, F. Boulanger, and S. Chabridon, "An Approach to Design Smart Grids and Their It System by Cosimulation," in *Proceedings of the MODELSWARD 2019: 7th International Conference on Model-Driven Engineering and Software Development* (SCITEPRESS, 2019), 372–379.
3. D. Bhor, K. Angappan, and K. M. Sivalingam, "A Co-Simulation Framework for Smart Grid Wide-Area Monitoring Networks," in *Proceedings of the 2014 Sixth International Conference on Communication Systems and Networks (COMSNETS)* (IEEE, 2014), 1–8.
4. P. Kannisto, V. Heikkilä, O. Hylli, M. Attar, S. Repo, and K. Systä, "Simces Platform for Modular Simulation: Featuring Platform Independence, Container Ecosystem, and Development Toolkit," *SoftwareX* 19 (2022): 101189, <https://doi.org/10.1016/j.softx.2022.101189>.
5. F. E. Abrahamsen, Y. Ai, and M. Cheffena, "Communication Technologies for Smart Grid: A Comprehensive Survey," *Sensors (Basel)* 21, no. 23 (2021): 8087, <https://doi.org/10.3390/s21238087>.
6. N. Javaid, N. Jan, and M. U. Javed, "An Adaptive Synthesis to Handle Imbalanced Big Data With Deep Siamese Network for Electricity Theft Detection in Smart Grids," *Journal of Parallel and Distributed Computing* 153 (2021): 44–52, <https://doi.org/10.1016/j.jpdc.2021.03.002>.
7. H. Lin, S. S. Veda, S. S. Shukla, L. Mili, and J. Thorp, "Geco: Global Event-Driven Co-Simulation Framework for Interconnected Power System and Communication Network," *IEEE Transactions on Smart Grid* 3, no. 3 (2012): 1444–1456.
8. EPRI, "Epri, Open Distribution System Simulator," (2025), <https://smartgrid.epri.com/SimulationTool.aspx>.
9. G. O. Troiano, H. S. Ferreira, F. C. Trindade, and L. F. Ochoa, "Co-Simulator of Power and Communication Networks Using Opendss and Omnet++," in *Proceedings of the 2016 IEEE Innovative Smart Grid Technologies-Asia (ISGT-Asia)* (IEEE, 2016), 1094–1099.
10. OMNeT++, "An overview of the OMNeT++ simulation environment," (2025), <https://omnetpp.org>.
11. A. Varga, and R. Hornig, *An overview of the OMNeT++ simulation environment*, ICST (Institute for Computer Sciences, Social-Informatics and Telecommunications Engineering) Brussels, BEL, *Proceedings of the 1st International Conference on Simulation Tools and Techniques for Communications* (Networks and Systems & Workshops, 2008).
12. E. O. Schweitzer, III, D. Whitehead, K. Fodero, and P. Robertson, "Merging SONET and Ethernet Communications for Power System Applications," *Journal of Reliable Power* 3, no. 2 (2012): 1–9.
13. M. Mirz, L. Razik, J. Dinkelbach, et al., "A Cosimulation Architecture for Power System, Communication, and Market in the Smart Grid," in *Complexity* 2018, no. 1 (Wiley Online Library, 2018), 7154031.

14. M. Albano, L. L. Ferreira, L. M. Pinho, and A. R. Alkhawaja, "Message-Oriented Middleware for Smart Grids," *Computer Standards & Interfaces* 38 (2015): 133–143, <https://doi.org/10.1016/j.csi.2014.08.002>.
15. N. Makarem, W. Bou Diab, I. Mougharbel, and N. Malouch, "On the Design of Efficient Congestion Control for the Constrained Application Protocol in Iot," *Computer Networks* 207 (2022): 108824, <https://doi.org/10.1016/j.comnet.2022.108824>.
16. L. Sun, Y. Chen, Q. Du, Q. Cheng, R. Ding, and Z. Liu, "Virtual Power Plant for Monitoring of Distributed Energy Resources Using Extensible Messaging and Presence Protocol," *Sustainable Energy, Grids and Networks* 38 (2024): 101365, <https://doi.org/10.1016/j.segan.2024.101365>.
17. P. G. Wagner, P. Birnstill, and J. Beyerer, "Dds Security+: Enhancing the Data Distribution Service With Tpm-Based Remote Attestation," in *Proceedings of the 19th International Conference on Availability, Reliability and Security. ARES '24* (Association for Computing Machinery, 2024).
18. C. Currier, C. Hummert, and D. Pawlaszczyk, eds., "Protocol Buffers," in *Mobile Forensics — The File Format Handbook: Common File Formats and File Systems Used in Mobile Devices* (Springer International Publishing, 2022), 223–260.
19. N. Narkhede, G. Shapira, and T. Palino, *Kafka: The Definitive Guide: Real-Time Data and Stream Processing at Scale* (O'Reilly Media, Inc., 2017).
20. M. H. Ansari, V. T. Vakili, and B. Bahrak, "Evaluation of Big Data Frameworks for Analysis of Smart Grids," *Journal of Big Data* 6, no. 1 (2019): 1–14.
21. H. Daki, A. El Hannani, A. Aqqal, A. Haidine, and A. Dahbi, "Big Data Management in Smart Grid: Concepts, Requirements and Implementation," *Journal of Big Data* 4, no. 1 (2017): 1–19.
22. P. Mell and T. Grance, "The Nist Definition of Cloud Computing," 2011.
23. E. N. Yilmaz, H. Polat, S. Oyucu, A. Aksoz, and A. Saygin, *6th International Istanbul Smart Grids and Cities Congress and Fair (ICSG), Data Storage in Smart Grid Systems* (IEEE, 2018), 110–113, <https://doi.org/10.1109/SGCF.2018.8408953>.
24. Z. Meng and Y. Wang, *IEEE 2nd International Conference on Electronic Technology, Communication and Information (ICETCI), Asymmetric Encryption Algorithms: Primitives and Applications* (IEEE, 2022), 876–881, <https://doi.org/10.1109/ICETCI55101.2022.9832032>.
25. Kubernetes, "Kubernetes Concepts and Architecture," (2025), <https://kubernetes.io/docs/concepts/architecture/>.
26. C. M. Software, "AMQP-CPP," (2013), <https://github.com/CopernicaMarketingSoftware/AMQP-CPP>.
27. Pika, "Pika," (2013), <https://github.com/pika/pika>.
28. RabbitMQ, "Quorum Queues — Rabbitmq," (2025), <https://www.rabbitmq.com/quorum-queues.html>.
29. Pyca, "Pyca/Cryptography: Cryptography a Package Designed to Expose Cryptographic Primitives and Recipes to Python Developers," (2025), <https://github.com/pyca/cryptography>.
30. Kubernetes, "Kubeadm," (2025), <https://kubernetes.io/docs/reference/setup-tools/kubeadm/>.
31. W. H. Kersting, "Radial Distribution Test Feeders," *IEEE Transactions on Power Systems* 6, no. 3 (1991): 975–985.
32. C. Rodriguez, M. Baez, F. Daniel, et al., "Rest Apis: A Large-Scale Analysis of Compliance With Principles and Best Practices," (2016), 21–39.
33. RabbitMQ, "Amqp 0–9–1 Model Explained," (2025), <https://www.rabbitmq.com/tutorials/amqp-concepts.html>.
34. B. Karimi, V. Namboodiri, and M. Jadliwala, "Scalable Meter Data Collection in Smart Grids Through Message Concatenation," *IEEE Transactions on Smart Grid* 6, no. 4 (2015): 1697–1706.
35. D. Hartmann, K. Wolter, and T. Krauss, "Ict Resilience Simulations in Small Confined Smart Distribution Grids," in *Proceeding of the Intelec 2013: 35th International Telecommunications Energy Conference, SMART POWER AND EFFICIENCY* (VDE, 2013), 1–6.
36. M. Jaber, N. Kouzayha, Z. Dawy, and A. Kayssi, "On Cellular Network Planning and Operation With m2m Signalling and Security Considerations," in *Proceedings of the 2014 IEEE International Conference on Communications Workshops (ICC)* (IEEE, 2014), 429–434.